

■ 3.9.6 Numerical Root Finding

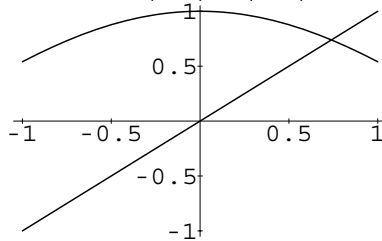
NRoots gives you a general way to find numerical approximations to the solutions of polynomial equations. Finding numerical solutions to more general equations, however, can be much more difficult, as discussed in Section 3.4.2. **FindRoot** gives you a way to search for a numerical solution to an arbitrary equation, or set of equations.

<code>FindRoot[<i>lhs</i>==<i>rhs</i>, {<i>x</i>, <i>x</i>₀}]</code>	search for a numerical solution to the equation <i>lhs</i> == <i>rhs</i> , starting with <i>x</i> = <i>x</i> ₀
<code>FindRoot[<i>lhs</i>==<i>rhs</i>, {<i>x</i>, {<i>x</i>₀, <i>x</i>₁}}]</code>	search for a solution using <i>x</i> ₀ and <i>x</i> ₁ as the first two values of <i>x</i> (this form must be used if symbolic derivatives of the equation cannot be found)
<code>FindRoot[<i>lhs</i>==<i>rhs</i>, {<i>x</i>, <i>x</i>_{start}, <i>x</i>_{min}, <i>x</i>_{max}}]</code>	search for a solution, stopping the search if <i>x</i> ever gets outside the range <i>x</i> _{min} to <i>x</i> _{max}
<code>FindRoot[{<i>eqn</i>₁, <i>eqn</i>₂, ...}, {<i>x</i>, <i>x</i>₀}, {<i>y</i>, <i>y</i>₀}, ...]</code>	search for a numerical solution to the simultaneous equations <i>eqn</i> _{<i>i</i>}

Numerical root finding.

The curves for $\cos(x)$ and x intersect at one point.

```
In[1]:= Plot[{Cos[x], x}, {x, -1, 1}]
```



This finds a numerical approximation to the value of x at which the intersection occurs. The 0 tells **FindRoot** what value of x to try first.

```
In[2]:= FindRoot[Cos[x] == x, {x, 0}]
```

```
Out[2]= {x -> 0.739085}
```

In trying to find a solution to your equation, **FindRoot** starts at the point you specify, and then progressively tries to get closer and closer to a solution. Even if your equations have several solutions, **FindRoot** always returns the first solution it finds. Which solution this is will depend on what starting point you chose. So long

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

as you start sufficiently close to a particular solution, `FindRoot` will always return that solution.

The equation $\sin(x) = 0$ has an infinite number of solutions of the form $x = n\pi$. If you start sufficiently close to a particular solution, `FindRoot` will give you that solution.

```
In[3]:= FindRoot[Sin[x] == 0, {x, 3}]
Out[3]= {x -> 3.14159}
```

If you start with $x = 6$, you get a numerical approximation to the solution $x = 2\pi$.

```
In[4]:= FindRoot[Sin[x] == 0, {x, 6}]
Out[4]= {x -> 6.28319}
```

This is what happens if `FindRoot` cannot find a solution to your equation.

```
In[5]:= FindRoot[Sin[x] == 2, {x, 1}]
FindRoot::convNewt:
  Newton's method failed to converge to the prescribed
  accuracy after 15 iterations.
Out[5]= {x -> 2.06095}
```

If you want `FindRoot` to search for complex solutions to this equation, then you have to give a complex starting value.

```
In[6]:= FindRoot[Sin[x] == 2, {x, I}]
Out[6]= {x -> 1.5708 + 1.31696 I}
```

You can give `FindRoot` bounds on the region in which you want it to look for solutions.

This tells `FindRoot` to try values of x starting at 1, but never going outside the region 0.5 to 1.5. In this case, `FindRoot` finds no solutions in the region you specified.

```
In[7]:= FindRoot[Sin[x] == 0, {x, 1, 0.5, 1.5}]
FindRoot::regext:
  Reached the point {-0.557408}
  which is outside the region {{0.5, 1.5}}.
Out[7]= FindRoot[Sin[x] == 0, {x, 1, 0.5, 1.5}]
```

Picking good starting points is crucial in getting useful answers from `FindRoot`. To know how to pick good starting points, you need to understand a little about how `FindRoot` actually works.

In the simplest case, `FindRoot` uses *Newton's method*. To find a solution to an equation of the form $f(x) = 0$, the method starts at x_0 , then uses knowledge of the derivative f' to take a sequence of steps towards a solution. Each new point x_n that it tries is found from the previous point x_{n-1} by the formula $x_n = x_{n-1} - f(x_{n-1})/f'(x_{n-1})$.

One important limitation of Newton's method is that it “gets stuck” if it ever gets to a point where the derivative of the function vanishes. You can usually avoid this problem by choosing starting points that have no special properties with respect to the equations you are trying to solve.

The derivative of $x^2 - 1$ is zero at the starting point $x = 0$. As a result **FindRoot** cannot decide whether to take its first step in the positive or the negative direction.

```
In[8]:= FindRoot[x^2 - 1 == 0, {x, 0}]
FindRoot::jsing:
  Encountered a singular Jacobian at the point {0.}
  . Try perturbing the initial point.

Out[8]= FindRoot[x^2 - 1 == 0, {x, 0}]
```

If you start at a random point, however, **FindRoot** will usually succeed in finding a solution.

```
In[9]:= FindRoot[x^2 - 1 == 0, {x, Random[]}]
Out[9]= {x -> 1.}
```

FindRoot uses versions of Newton's method in many cases. Especially when there are several variables, the precise set of starting points which lead to a particular solution can become extremely complicated. The best policy is to try and start as close to the solution as possible, and to avoid any "special points".

This finds a solution to a set of simultaneous equations. It is a good idea to avoid taking the starting values for x and y to be equal, or in any other way "special".

```
In[10]:= FindRoot[{Sin[x] == Cos[y], x + y == 1},
  {x, .1}, {y, .2}]
Out[10]= {x -> 1.2854, y -> -0.285398}
```

If the functions that appear in your equations are sufficiently simple, then *Mathematica* will be able to find their derivatives symbolically. In all the examples of **FindRoot** that we have used so far, this is possible. As a result, **FindRoot** can use the formula for Newton's method directly.

If, on the other hand, **FindRoot** has to estimate the derivative of your functions numerically, then it must take another approach. In simple cases, the approach it uses is based on the "secant method". One feature of this method is that to get it started, you have to specify not just the first value to try, but rather the first *two* values.

This specifies the first two values of x to try.

```
In[11]:= FindRoot[Cos[x] == x, {x, {0, 1}}]
Out[11]= {x -> 0.739085}
```

If *Mathematica* cannot get an explicit formula for the function that appears in your equation, you *have to* specify the first two values to try. Here **FindRoot** finds a zero of the Riemann zeta function.

```
In[12]:= FindRoot[Zeta[1/2 + I t] == 0, {t, {12, 13}}]
Out[12]= {t -> 14.1347 - 4.42626 10-7 I}
```

If you are finding a root of a function of one variable, and the first two points you tell **FindRoot** to try give values of the function with opposite signs, then **FindRoot** is guaranteed to find a root. (This is true so long as your function satisfies some basic continuity conditions.)

AccuracyGoal	6	the accuracy with which functions must be zero in order to accept a candidate solution
WorkingPrecision	<code>Precision[1.]</code>	the number of digits of precision to keep in internal computations
MaxIterations	15	the maximum number of steps to take in trying to find a solution

Options for `FindRoot`.

There are several options you can use to control the operation of `FindRoot`. First, you can set `MaxIterations` to specify the maximum number of steps that `FindRoot` should use in attempting to find a solution. Even if `FindRoot` does not successfully find a solution in the number of steps you specify, it returns the most recent values it has tried. If you want to continue the search, you can then give these values as starting points.

To work out whether it has found an acceptable root, `FindRoot` evaluates your function and sees whether the result is zero to within the accuracy specified by the option `AccuracyGoal`. `FindRoot` will always print a warning message if it does not find a solution to within the specified accuracy. In doing internal computations, `FindRoot` uses the precision specified by the option `WorkingPrecision`.

This specifies that the zeta function needs to be zero to 10 digit accuracy at the solution. The Riemann hypothesis asserts that the imaginary part of t should be exactly zero at a root.

```
In[13]:= FindRoot[Zeta[1/2 + I t] == 0, {t, {12, 13}},
               AccuracyGoal -> 10]
```

```
Out[13]= {t -> 14.1347 + 1.79056 10-11 I}
```